

CO4 - AT2

Java Lab Practice Record

Applets, AWT Components and Layout Managers

Name: AKILESHWARAN A

Register Number: 192511534

Department:CSE

Date: 23/08/26

Index

S.No	Title	Page
1	Applet Life Cycle	
2	Generating Graphic Images Using Applet	
3	Calculator Application Using Applet Parameters	
4	Student Profile Using AWT Components	
5	GridLayout with Event Handling	
6	GridBagLayout with Event Handling	
7	BorderLayout with Event Handling	
8	CardLayout with Event Handling	

Question 1 – Applet Life Cycle

Aim:

To develop a Java Applet to demonstrate the Applet Life Cycle methods: init(), start(), paint(), stop(), and destroy(), displaying a suitable message whenever each method is invoked.

Program (LifeCycle.java):

```
import java.applet.Applet;
import java.awt.Graphics;

public class LifeCycle extends Applet {

    public void init() {
        System.out.println("init() method called");
    }

    public void start() {
        System.out.println("start() method called");
    }

    public void paint(Graphics g) {
        g.drawString("Applet Life Cycle", 50, 50);
        g.drawString("init() -> start() -> paint()", 50, 80);
        System.out.println("paint() method called");
    }

    public void stop() {
        System.out.println("stop() method called");
    }

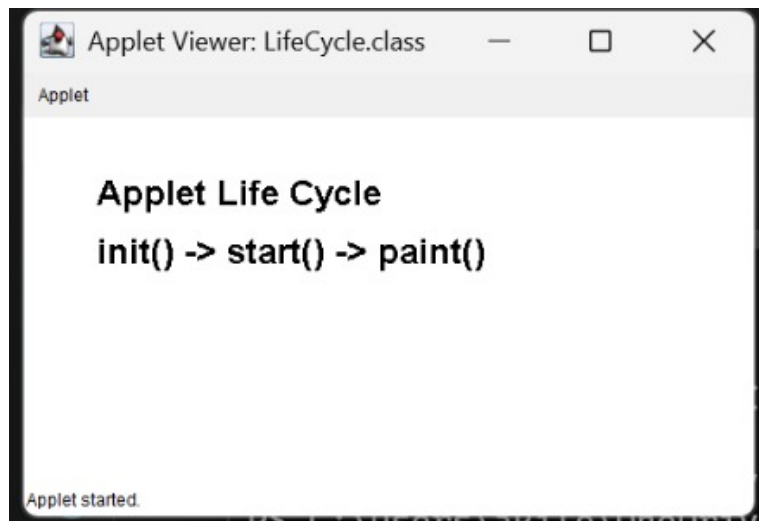
    public void destroy() {
        System.out.println("destroy() method called");
    }

}
```

HTML (LifeCycle.html):

```
<html>
<head>
<title>Applet Life Cycle</title>
</head>
<body>
<applet code="LifeCycle.class" width="400" height="200">
</applet>
</body>
</html>
```

Output:



Result:

Thus, the Java Applet program to demonstrate the Applet Life Cycle was executed successfully, and the sequence `init()` -> `start()` -> `paint()` was observed.

Question 2 – Generating Graphic Images Using Applet

Aim:

To develop a Java Applet that uses the Graphics class to draw a graphical scene consisting of a house, sun, trees, road, and clouds using methods such as drawLine(), drawRect(), drawOval(), fillRect(), fillOval(), and fillPolygon().

Program (Scenery.java):

```
import java.applet.Applet;
import java.awt.Graphics;
import java.awt.Color;

public class Scenery extends Applet {

    public void paint(Graphics g) {
        // Sky background
        g.setColor(new Color(135, 206, 235));
        g.fillRect(0, 0, 500, 300);

        // Sun
        g.setColor(Color.YELLOW);
        g.fillOval(400, 30, 60, 60);

        // Clouds
        g.setColor(Color.WHITE);
        g.fillOval(60, 40, 50, 30);
        g.fillOval(90, 30, 50, 30);
        g.fillOval(120, 40, 50, 30);

        g.fillOval(220, 70, 40, 25);
        g.fillOval(245, 60, 40, 25);
        g.fillOval(270, 70, 40, 25);

        // Ground
        g.setColor(new Color(34, 139, 34));
        g.fillRect(0, 220, 500, 80);

        // Road
        g.setColor(Color.DARK_GRAY);
        g.fillRect(0, 260, 500, 40);
        g.setColor(Color.WHITE);
        for (int x = 10; x < 500; x += 40) {
            g.fillRect(x, 278, 20, 4);
        }

        // House - base
        g.setColor(new Color(205, 133, 63));
        g.fillRect(150, 150, 100, 70);

        // House - roof
        g.setColor(Color.RED);
        int[] roofX = {140, 200, 260};
        int[] roofY = {150, 100, 150};
        g.fillPolygon(roofX, roofY, 3);

        // House - door
        g.setColor(new Color(101, 67, 33));
        g.fillRect(185, 190, 25, 30);

        // House - window
```

```

        g.setColor(Color.CYAN);
        g.fillRect(160, 165, 20, 20);
        g.setColor(Color.BLACK);
        g.drawRect(160, 165, 20, 20);

        // Trees
        drawTree(g, 350, 220);
        drawTree(g, 60, 220);
    }

    private void drawTree(Graphics g, int x, int y) {
        // Trunk
        g.setColor(new Color(139, 69, 19));
        g.fillRect(x, y - 40, 10, 40);

        // Leaves
        g.setColor(Color.GREEN);
        g.fillOval(x - 20, y - 80, 50, 50);
    }
}

```

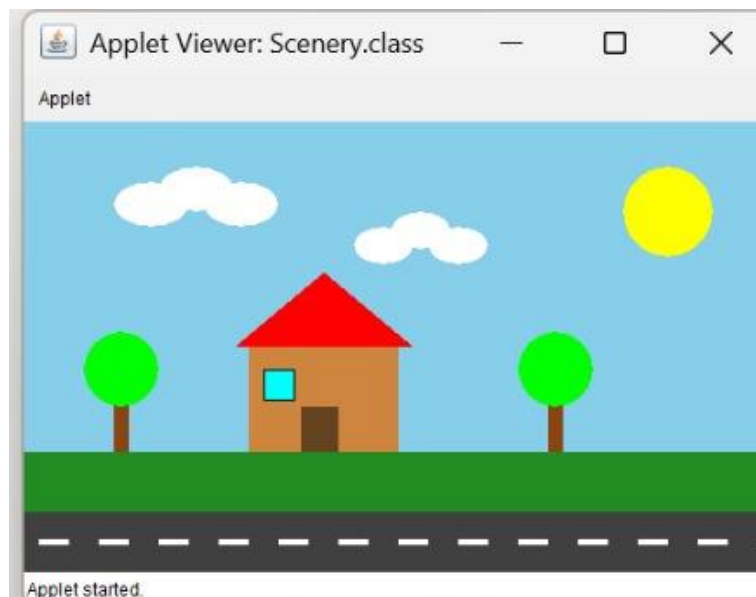
HTML (Scenery.html):

```

<html>
<head>
<title>Graphic Scenery</title>
</head>
<body>
<applet code="Scenery.class" width="500" height="300">
</applet>
</body>
</html>

```

Output:



Result:

Thus, the Java Applet program to generate a graphical scene using the Graphics class was executed successfully.

Question 3 – Calculator Application Using Applet Parameters

Aim:

To develop a Calculator Applet that accepts two numbers and an arithmetic operator through HTML/Applet parameters and performs addition, subtraction, multiplication, and division using the `getParameter()` method.

Program (Calculator.java):

```
import java.applet.Applet;
import java.awt.Graphics;
import java.awt.Font;

public class Calculator extends Applet {

    double num1, num2, result;
    String operator;

    public void init() {
        String p1 = getParameter("num1");
        String p2 = getParameter("num2");
        operator = getParameter("operator");

        num1 = Double.parseDouble(p1);
        num2 = Double.parseDouble(p2);

        switch (operator) {
            case "+":
                result = num1 + num2;
                break;
            case "-":
                result = num1 - num2;
                break;
            case "*":
                result = num1 * num2;
                break;
            case "/":
                if (num2 != 0) {
                    result = num1 / num2;
                } else {
                    result = 0;
                }
                break;
            default:
                result = 0;
        }
    }

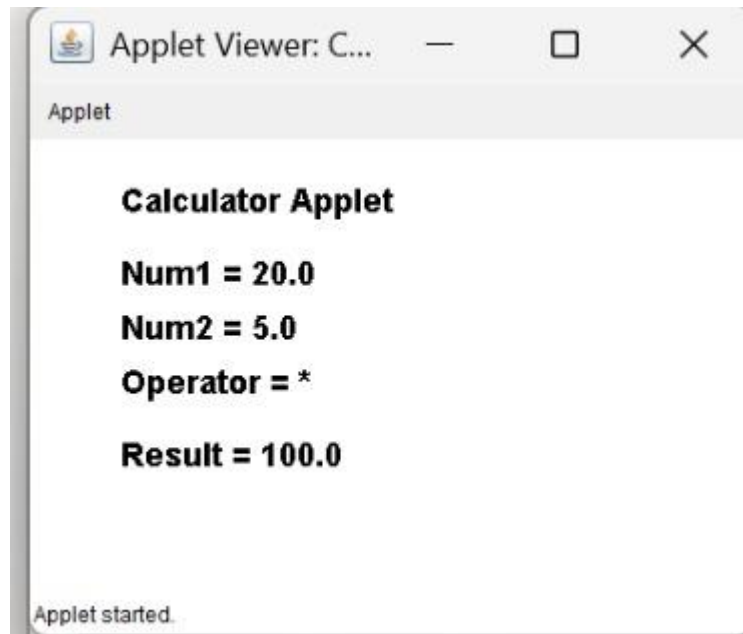
    public void paint(Graphics g) {
        g.setFont(new Font("Arial", Font.BOLD, 18));
        g.drawString("Calculator Applet", 50, 40);
        g.drawString("Num1 = " + num1, 50, 80);
        g.drawString("Num2 = " + num2, 50, 110);
        g.drawString("Operator = " + operator, 50, 140);
        g.drawString("Result = " + result, 50, 180);
    }
}
```

HTML (Calculator.html):

```
<html>
```

```
<head>
<title>Calculator Applet</title>
</head>
<body>
<applet code="Calculator.class" width="400" height="250">
<param name="num1" value="20">
<param name="num2" value="5">
<param name="operator" value="*">
</applet>
</body>
</html>
```

Output:



Result:

Thus, the Calculator Applet using `getParameter()` to retrieve values from the HTML page was executed successfully, and the multiplication of 20.0 and 5.0 correctly produced a result of 100.0.

Question 4 – Student Profile Using AWT Components

Aim:

To develop a Student Profile Application using Java AWT components (Label, TextField, Checkbox, CheckboxGroup, Choice, TextArea, Button, List) that accepts student details and displays them in a List when the Generate Report button is clicked.

Program (StudentProfile.java):

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class StudentProfile extends Applet implements ActionListener {

    Label nameLabel, regLabel, deptLabel, yearLabel, skillsLabel;
    TextField nameField, regField;
    CheckboxGroup genderGroup;
    Checkbox male, female;
    Choice deptChoice, yearChoice;
    TextArea skillsArea, outputArea;
    Button generateBtn;
    List reportList;

    public void init() {
        setLayout(new FlowLayout());

        nameLabel = new Label("Name:");
        nameField = new TextField(15);

        regLabel = new Label("Register Number:");
        regField = new TextField(15);

        genderGroup = new CheckboxGroup();
        male = new Checkbox("Male", genderGroup, true);
        female = new Checkbox("Female", genderGroup, false);

        deptLabel = new Label("Department:");
        deptChoice = new Choice();
        deptChoice.add("CSE");
        deptChoice.add("ECE");
        deptChoice.add("MECH");
        deptChoice.add("CIVIL");
        deptChoice.add("IT");

        yearLabel = new Label("Year:");
        yearChoice = new Choice();
        yearChoice.add("1st Year");
        yearChoice.add("2nd Year");
        yearChoice.add("3rd Year");
        yearChoice.add("4th Year");

        skillsLabel = new Label("Skills:");
        skillsArea = new TextArea(3, 20);

        generateBtn = new Button("Generate Report");
        generateBtn.addActionListener(this);

        reportList = new List(6, false);
    }
}
```

```

        add(nameLabel); add(nameField);
        add(regLabel); add(regField);
        add(male); add(female);
        add(deptLabel); add(deptChoice);
        add(yearLabel); add(yearChoice);
        add(skillsLabel); add(skillsArea);
        add(generateBtn);
        add(reportList);
    }

    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == generateBtn) {
            reportList.removeAll();

            String gender = genderGroup.getSelectedCheckbox().getLabel();

            reportList.add("Name: " + nameField.getText());
            reportList.add("Register No: " + regField.getText());
            reportList.add("Gender: " + gender);
            reportList.add("Department: " + deptChoice.getSelectedItem());
            reportList.add("Year: " + yearChoice.getSelectedItem());
            reportList.add("Skills: " + skillsArea.getText());
        }
    }
}

```

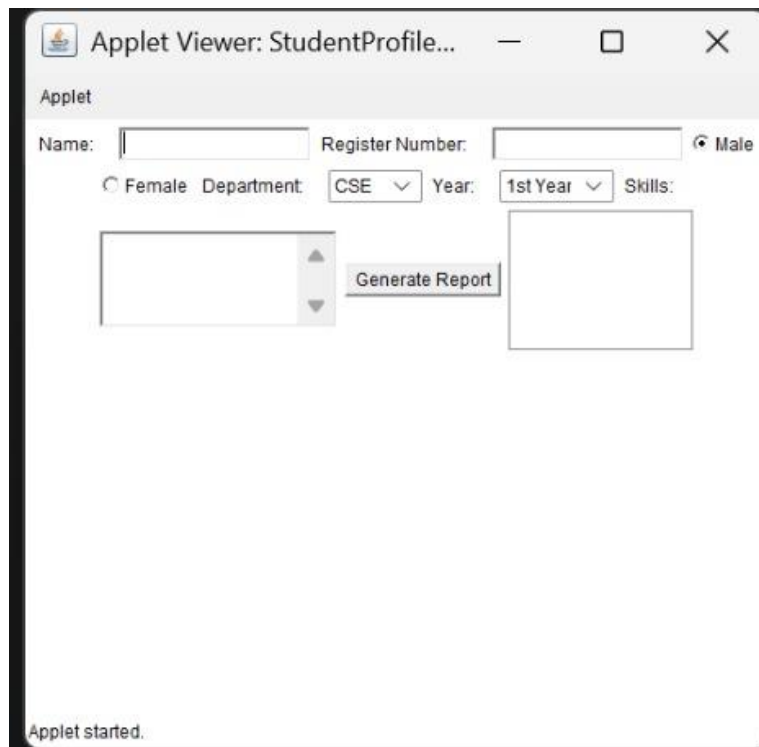
HTML (StudentProfile.html):

```

<html>
<head>
<title>Student Profile</title>
</head>
<body>
<applet code="StudentProfile.class" width="500" height="400">
</applet>
</body>
</html>

```

Output:



Result:

Thus, the Student Profile Application demonstrating multiple AWT components and event handling was executed successfully.

Question 5 – GridLayout with Event Handling

Aim:

To develop a Java AWT application using GridLayout to create a calculator interface with digit and operator buttons arranged in a grid, implementing event handling using ActionListener.

Program (GridCalculator.java):

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class GridCalculator extends Applet implements ActionListener {

    TextField display;
    double num1 = 0, num2 = 0, result = 0;
    char operator;
    boolean startNewNumber = true;

    public void init() {
        setLayout(new BorderLayout());

        display = new TextField();
        display.setEditable(false);
        add(display, BorderLayout.NORTH);

        Panel buttonPanel = new Panel();
        buttonPanel.setLayout(new GridLayout(4, 4, 5, 5));

        String[] buttons = {
            "7", "8", "9", "+",
            "4", "5", "6", "-",
            "1", "2", "3", "*",
            "0", "=", "C", "/"
        };

        for (String label : buttons) {
            Button b = new Button(label);
            b.addActionListener(this);
            buttonPanel.add(b);
        }

        add(buttonPanel, BorderLayout.CENTER);
    }

    public void actionPerformed(ActionEvent e) {
        String cmd = e.getActionCommand();

        if (cmd.charAt(0) >= '0' && cmd.charAt(0) <= '9') {
            if (startNewNumber) {
                display.setText(cmd);
                startNewNumber = false;
            } else {
                display.setText(display.getText() + cmd);
            }
        } else if (cmd.equals("C")) {
            display.setText("");
            num1 = 0;
            num2 = 0;
            result = 0;
            startNewNumber = true;
        }
    }
}
```

```

    } else if (cmd.equals("=")) {
        num2 = Double.parseDouble(display.getText());
        switch (operator) {
            case '+': result = num1 + num2; break;
            case '-': result = num1 - num2; break;
            case '*': result = num1 * num2; break;
            case '/':
                if (num2 != 0) result = num1 / num2;
                else result = 0;
                break;
        }
        display.setText(String.valueOf(result));
        startNewNumber = true;
    } else {
        num1 = Double.parseDouble(display.getText());
        operator = cmd.charAt(0);
        startNewNumber = true;
    }
}
}

```

HTML (GridCalculator.html):

```

<html>
<head>
<title>Grid Calculator</title>
</head>
<body>
<applet code="GridCalculator.class" width="300" height="300">
</applet>
</body>
</html>

```

Output:



Result:

Thus, the GridLayout-based calculator application with button event handling was executed successfully.

Question 6 – GridBagLayout with Event Handling

Aim:

To develop a Java AWT Student Registration Form using GridBagLayout with GridBagConstraints to align Name, Register Number, Department, Email, Gender, and Year fields, along with Submit and Reset button event handling.

Program (RegistrationForm.java):

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class RegistrationForm extends Applet implements ActionListener {

    TextField nameField, regField, deptField, emailField;
    CheckboxGroup genderGroup;
    Checkbox male, female;
    Choice yearChoice;
    Button submitBtn, resetBtn;
    Label statusLabel;

    public void init() {
        setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 5, 5, 5);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Name
        gbc.gridx = 0; gbc.gridy = 0;
        add(new Label("Name:"), gbc);
        nameField = new TextField(15);
        gbc.gridx = 1; gbc.gridy = 0;
        add(nameField, gbc);

        // Register Number
        gbc.gridx = 0; gbc.gridy = 1;
        add(new Label("Register Number:"), gbc);
        regField = new TextField(15);
        gbc.gridx = 1; gbc.gridy = 1;
        add(regField, gbc);

        // Department
        gbc.gridx = 0; gbc.gridy = 2;
        add(new Label("Department:"), gbc);
        deptField = new TextField(15);
        gbc.gridx = 1; gbc.gridy = 2;
        add(deptField, gbc);

        // Email
        gbc.gridx = 0; gbc.gridy = 3;
        add(new Label("Email:"), gbc);
        emailField = new TextField(15);
        gbc.gridx = 1; gbc.gridy = 3;
        add(emailField, gbc);

        // Gender
        gbc.gridx = 0; gbc.gridy = 4;
        add(new Label("Gender:"), gbc);
        Panel genderPanel = new Panel();
```

```

genderGroup = new CheckboxGroup();
male = new Checkbox("Male", genderGroup, true);
female = new Checkbox("Female", genderGroup, false);
genderPanel.add(male);
genderPanel.add(female);
gbc.gridx = 1; gbc.gridy = 4;
add(genderPanel, gbc);

// Year
gbc.gridx = 0; gbc.gridy = 5;
add(new Label("Year:"), gbc);
yearChoice = new Choice();
yearChoice.add("1st Year");
yearChoice.add("2nd Year");
yearChoice.add("3rd Year");
yearChoice.add("4th Year");
gbc.gridx = 1; gbc.gridy = 5;
add(yearChoice, gbc);

// Buttons
submitBtn = new Button("Submit");
submitBtn.addActionListener(this);
gbc.gridx = 0; gbc.gridy = 6;
add(submitBtn, gbc);

resetBtn = new Button("Reset");
resetBtn.addActionListener(this);
gbc.gridx = 1; gbc.gridy = 6;
add(resetBtn, gbc);

// Status label
statusLabel = new Label(" ");
gbc.gridx = 0; gbc.gridy = 7;
gbc.gridwidth = 2;
add(statusLabel, gbc);
}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == submitBtn) {
        String gender = genderGroup.getSelectedCheckbox().getLabel();
        statusLabel.setText("Registered: " + nameField.getText() + ", " +
            regField.getText() + ", " + deptField.getText() + ", " +
            emailField.getText() + ", " + gender + ", " +
            yearChoice.getSelectedItem());
    } else if (e.getSource() == resetBtn) {
        nameField.setText("");
        regField.setText("");
        deptField.setText("");
        emailField.setText("");
        male.setState(true);
        yearChoice.select(0);
        statusLabel.setText(" ");
    }
}
}

```

HTML (RegistrationForm.html):

```

<html>
<head>
<title>Student Registration Form</title>
</head>

```



```
<body>
<applet code="RegistrationForm.class" width="450" height="350">
</applet>
</body>
</html>
```

Output:

Applet Viewer: Registr... — □ ×

Applet

Name:

Register Number:

Department:

Email:

Gender: ☒ Male ☐ Female

Year: ▾

Registered: akil, 192611, cse, efj8, Male, 1st Y

Applet started.

Result:

Thus, the Student Registration Form using GridBagLayout was executed successfully; the Submit button correctly displayed the entered registration details.

Question 7 – BorderLayout with Event Handling

Aim:

To develop a Java AWT application using BorderLayout, dividing the window into NORTH (Header), SOUTH (Buttons), EAST (Options), WEST (Menu), and CENTER (Display Area) regions, with event handling to update the center display area.

Program (BorderDemo.java):

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class BorderDemo extends Applet implements ActionListener {

    Label centerLabel;
    Button homeBtn, aboutBtn, contactBtn;
    Button optionBtn1, optionBtn2;
    Button menuBtn1, menuBtn2;

    public void init() {
        setLayout(new BorderLayout());

        Label header = new Label("Welcome to BorderLayout Demo", Label.CENTER);
        add(header, BorderLayout.NORTH);

        Panel southPanel = new Panel();
        homeBtn = new Button("Home");
        aboutBtn = new Button("About");
        contactBtn = new Button("Contact");
        homeBtn.addActionListener(this);
        aboutBtn.addActionListener(this);
        contactBtn.addActionListener(this);
        southPanel.add(homeBtn);
        southPanel.add(aboutBtn);
        southPanel.add(contactBtn);
        add(southPanel, BorderLayout.SOUTH);

        Panel eastPanel = new Panel();
        eastPanel.setLayout(new GridLayout(2, 1));
        optionBtn1 = new Button("Option 1");
        optionBtn2 = new Button("Option 2");
        optionBtn1.addActionListener(this);
        optionBtn2.addActionListener(this);
        eastPanel.add(optionBtn1);
        eastPanel.add(optionBtn2);
        add(eastPanel, BorderLayout.EAST);

        Panel westPanel = new Panel();
        westPanel.setLayout(new GridLayout(2, 1));
        menuBtn1 = new Button("Menu 1");
        menuBtn2 = new Button("Menu 2");
        menuBtn1.addActionListener(this);
        menuBtn2.addActionListener(this);
        westPanel.add(menuBtn1);
        westPanel.add(menuBtn2);
        add(westPanel, BorderLayout.WEST);

        centerLabel = new Label("Content Display Area", Label.CENTER);
        add(centerLabel, BorderLayout.CENTER);
    }
}
```

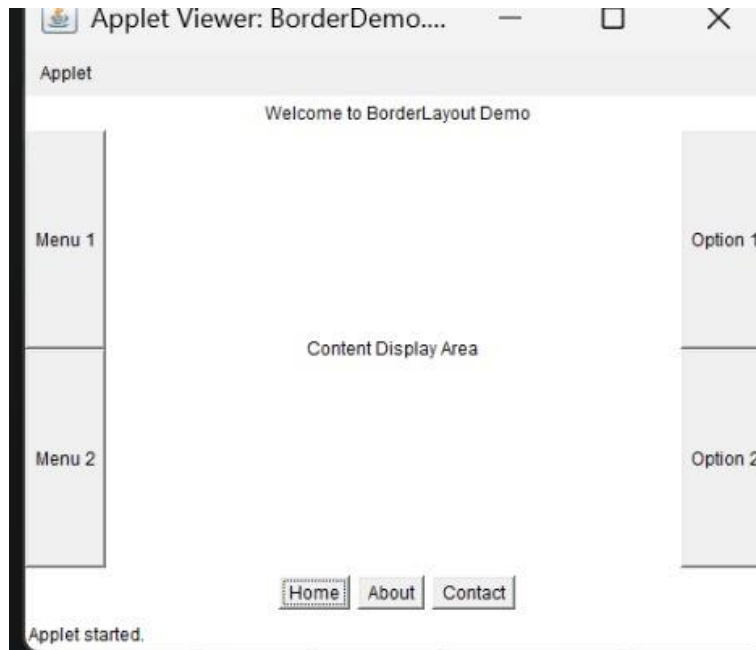
```
}

public void actionPerformed(ActionEvent e) {
    String cmd = e.getActionCommand();
    centerLabel.setText("You clicked: " + cmd);
}
}
```

HTML (BorderDemo.html):

```
<html>
<head>
<title>BorderLayout Demo</title>
</head>
<body>
<applet code="BorderDemo.class" width="500" height="350">
</applet>
</body>
</html>
```

Output:



Result:

Thus, the BorderLayout application demonstrating all five layout regions and event-driven programming was executed successfully.

Question 8 – CardLayout with Event Handling

Aim:

To develop a Java AWT application using CardLayout to create a multi-page interface with Student Information, Academic Information, and Confirmation cards, using Next, Previous, and Submit buttons to navigate between cards.

Program (CardDemo.java):

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class CardDemo extends Applet implements ActionListener {

    CardLayout cardLayout;
    Panel cardPanel;

    TextField nameField, regField;
    TextField deptField, yearField;
    Label confirmLabel;

    Button nextBtn1, nextBtn2, prevBtn2, submitBtn3, prevBtn3;

    public void init() {
        setLayout(new BorderLayout());
        cardLayout = new CardLayout();
        cardPanel = new Panel();
        cardPanel.setLayout(cardLayout);

        Panel card1 = new Panel(new GridLayout(3, 2));
        card1.add(new Label("Name:"));
        nameField = new TextField(15);
        card1.add(nameField);
        card1.add(new Label("Register Number:"));
        regField = new TextField(15);
        card1.add(regField);
        nextBtn1 = new Button("Next");
        nextBtn1.addActionListener(this);
        card1.add(new Label(""));
        card1.add(nextBtn1);

        Panel card2 = new Panel(new GridLayout(3, 2));
        card2.add(new Label("Department:"));
        deptField = new TextField(15);
        card2.add(deptField);
        card2.add(new Label("Year:"));
        yearField = new TextField(15);
        card2.add(yearField);
        prevBtn2 = new Button("Previous");
        nextBtn2 = new Button("Next");
        prevBtn2.addActionListener(this);
        nextBtn2.addActionListener(this);
        card2.add(prevBtn2);
        card2.add(nextBtn2);

        Panel card3 = new Panel(new BorderLayout());
        confirmLabel = new Label("Click Submit to confirm", Label.CENTER);
        card3.add(confirmLabel, BorderLayout.CENTER);
        Panel card3Buttons = new Panel();
```

```

        prevBtn3 = new Button("Previous");
        submitBtn3 = new Button("Submit");
        prevBtn3.addActionListener(this);
        submitBtn3.addActionListener(this);
        card3Buttons.add(prevBtn3);
        card3Buttons.add(submitBtn3);
        card3.add(card3Buttons, BorderLayout.SOUTH);

        cardPanel.add(card1, "Student Information");
        cardPanel.add(card2, "Academic Information");
        cardPanel.add(card3, "Confirmation");

        add(cardPanel, BorderLayout.CENTER);
    }

    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();

        if (src == nextBtn1) {
            cardLayout.show(cardPanel, "Academic Information");
        } else if (src == prevBtn2) {
            cardLayout.show(cardPanel, "Student Information");
        } else if (src == nextBtn2) {
            cardLayout.show(cardPanel, "Confirmation");
        } else if (src == prevBtn3) {
            cardLayout.show(cardPanel, "Academic Information");
        } else if (src == submitBtn3) {
            confirmLabel.setText("Registered: " + nameField.getText() + ", " +
                                regField.getText() + ", " + deptField.getText() + ", " +
                                yearField.getText());
        }
    }
}

```

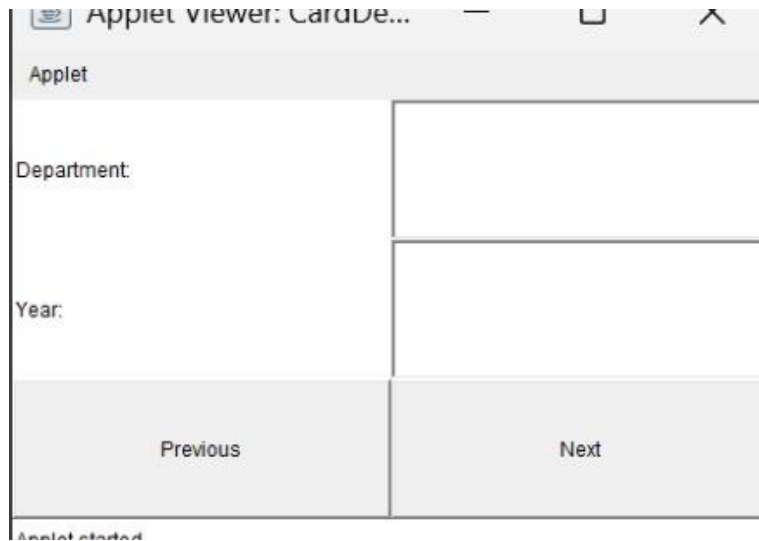
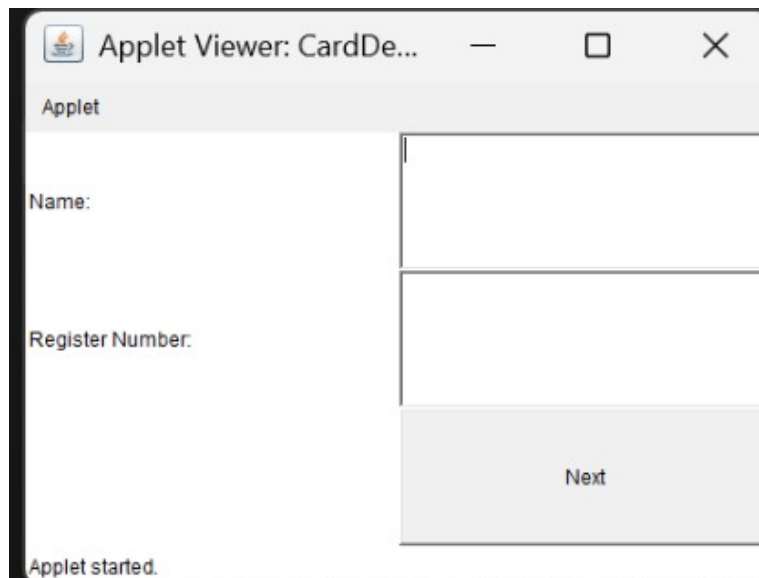
HTML (CardDemo.html):

```

<html>
<head>
<title>CardLayout Demo</title>
</head>
<body>
<applet code="CardDemo.class" width="450" height="250">
</applet>
</body>
</html>

```

Output:



Result:

Thus, the CardLayout application demonstrating multiple screens within a single window (Student Information -> Academic Information -> Confirmation) was executed successfully.